Intro to Data Science - Lab 9

Copyright 2025, Jeffrey Stanton and Jeffrey Saltz Please do not post online.

Week 9 - Supervised Data Mining

```
# Enter your name here: Tim Horne
```

Please include nice comments.

Instructions:

Run the necessary code on your own instance of R-Studio.

Attribution statement: (choose only one and delete the rest)

```
# I, I did this lab assignment by myself, with help from the book and the professor.
```

Supervised data mining/machine learning is the most prevalent form of data mining as it allows for the prediction of new cases in the future. In this lab we will look at two extremely popular supervised data mining methods called Rout. We will create a supervised model by training it on fraud data that they already have. Then they will deploy the model into the field. As new input data arrives the model predicts whether it seems fraudulent and flags those transactions where that probability is high.

In these exercises we will work with a built in data set called **GermanCredit**. This data set is in the " caret " package so we will need that and the " kernlab " package to be installed and library'ed before running the following:

```
data("GermanCredit")
subCreditC <- GermanCredit[,1:10]
str(subCreditC)

#install.packages("caret")
#install.packages("kernlab")

library(caret)

## Warning: package 'caret' was built under R version 4.1.3

## Loading required package: ggplot2

## Warning: package 'ggplot2' was built under R version 4.1.2

## Loading required package: lattice

## Registered S3 method overwritten by 'future':
##   method               from
##   all.equal.connection parallelly

library(kernlab)

## Warning: package 'kernlab' was built under R version 4.0.3

##
## Attaching package: 'kernlab'

##
## The following object is masked from 'package:ggplot2':
##
##     alpha

data("GermanCredit")
subCreditC <- GermanCredit[,1:10]
str(subCreditC)

##  'data.frame':   1000 obs. of  10 variables:
##  $ Duration     : int  6 48 12 42 24 36 24 36 12 30 ...
##  $ Amount       : int  1169 5951 2096 7882 4870 9055 2835 6948 3059 5234 ...
##  $ InstallmentRatePercentage: int  4 2 2 2 3 2 3 2 1 4 ...
##  $ ResidenceDuration        : int  4 2 3 4 4 2 4 2 1 4 ...
##  $ Age          : int  67 22 49 45 53 35 53 35 61 28 ...
##  $ NumberExistingCredits    : int  2 1 1 1 2 1 1 1 1 2 ...
##  $ NumberPeopleMaintenance  : int  1 1 2 2 2 2 1 2 1 1 ...
##  $ Telephone    : num  0 1 1 1 1 0 1 1 1 1 ...
##  $ ForeignWorker: num  1 1 1 1 1 1 1 1 1 1 ...
##  $ Class        : Factor w/ 2 levels "Bad","Good": 2 1 2 2 1 2 2 2 2 2 ...

1. Examine the data structure that **str()** reveals. Also use the **help()** command to learn more about the **GermanCredit** data set. Summarize what you see in a comment.

```
#GermanCredit
# This data has 1000 obs. of 10 variables, 9 of them are int and one is a factor.
```

2. Use the **createDataPartition()** function to generate a list of cases to include in the training data. This function is conveniently provided by caret and allows one to directly control the number of training cases. It also ensures that the training cases are balanced with respect to the outcome variable. Try this:

```
trainList <- createDataPartition(y=subCreditC$Class,p=.90,list=FALSE)

trainList <- createDataPartition(y=subCreditC$Class,p=.90,list=FALSE)
head(trainList)

##
##     Resample1
## [1,         1
## [2,         2
## [3,         3
## [4,         4
## [5,         5
## [6,         6
table(subCreditC$Class)

##
##  Bad Good
##  300  700
```

3. Examine the contents of **trainList** to make sure that it is a list of case numbers. With **p=0.90**, it should have 450 case numbers in it.

```
#trainList
```

4. What is **trainList**? What do the elements in **trainList** represent? Which attribute is balanced in the **trainList** dataset?

5. Use **trainList** and the square brackets notation to create a training data set called " trainSet " from the **subCreditC** data frame. Look at the structure of trainSet to make sure it has all of the same variables as **subCreditC**. The **trainSet** structure should be a data frame with **450 rows and 10 columns**.

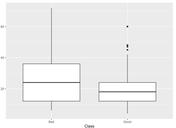
```
trainSet <- subCreditC[trainList, ]
#dim(trainSet)
```

6. Use **trainList** and the square brackets notation to create a testing data set called " testSet " from the subCreditC data frame. The **testSet** structure should be a data frame with **550 rows and 10 columns** and should be a completely different set of cases than **trainSet**.

```
testSet <- subCreditC[-trainList,]
```

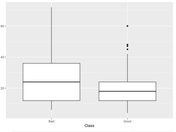
7. Create and interpret boxplots of all the predictor variables in relation to the outcome variable **(Class)**.

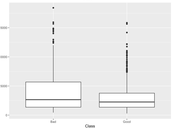
```
library(ggplot2)
ggplot(data = trainSet, aes(x=Class, y= Duration))+
  geom_boxplot()
```

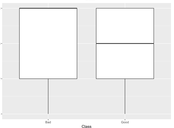


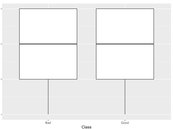

```
nice = colnames(trainSet)
nice <- nice[nice!='Class']

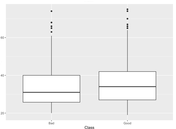
for (d in nice)
  print(ggplot(data = trainSet, aes(x=Class, y= .data[[d]]))+
  geom_boxplot())
```

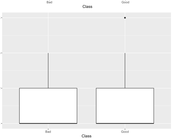


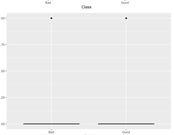


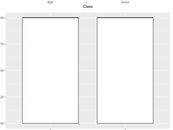


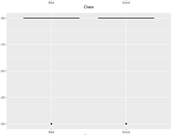












8. Train a support vector machine with the **ksvm()** function from the **kernlab** package. Make sure that you have installed and library'ed the **kernlab** package. Have the data set be **trainSet** and have **kcore=** to 3 cross **validations** 2000: `type = "prob",prob.model = TRUE`)

```
ksvm(Class~., data = trainSet, C=5, cross=3, prob.model= TRUE)

##
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc  (classification)
##  parameter : cost C = 5
##
## Gaussian Radial Basis kernel function.
##  Hyperparameter : sigma = 0.0074477566148898
##
## Number of Support Vectors : 523
##
##
## Objective Function Value = -1943.58
## Training error : 0.21429
## Cross validation error : 0.282569
## Probability model included.

# Either I mess the model all try harder to classify every point correctly (risking over fitting)
# I did cross validation mean the data is split into 3 parts - 2 used for training and one for testing. repeate
# I chose to activate model estimate

9. Examine the ksvm output object. In particular, look at the **cross-validation error** for an initial indication of model quality. Add a comment that gives your opinion on whether this is a good model.

svm_out <- ksvm(Class~., data = trainSet, C=5, cross=3, prob.model= TRUE)
print(svm_out)

##
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc  (classification)
##  parameter : cost C = 5
##
## Gaussian Radial Basis kernel function.
##  Hyperparameter : sigma = 0.0084151962978
##
## Number of Support Vectors : 525
##
##
## Objective Function Value = -2064.944
## Training error : 0.245714
## Cross validation error : 0.287567
## Probability model included.

#Training error : 0.24571
#Cross validation error : 0.287567

10. Predict the training cases using the **predict()** command

svm_pred <- predict(svm_out, testSet)

11. Examine the predicted out object with **str()**. Then, calculate a **confusion matrix** using the **table()** function.

str(svm_pred)

##  Factor w/ 2 levels "Bad","Good": 2 2 2 2 2 2 2 1 1 2 ...

table(testSet$Class, svm_pred)

##
##      svm_pred
##       Bad Good
##  Bad   75   45
##  Good   7  423

12. Interpret the confusion matrix and in particular calculate the overall **accuracy** of the model. The **diag()** command can be applied to the results of the table command you ran in the previous step. You can also use **sum()** to get the total of all four cells.

confusion_m <- table(testSet$Class, svm_pred)
# accuracy = correctly classified cases / total number of cases
sum(diag(confusion_m))/sum(confusion_m)

## [1] 0.76

13. Check you calculation with the **confusionMatrix()** function in the **caret** package.

confusionMatrix(svm_pred, testSet$Class)

##
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  Bad Good
##       Bad   75   45
##       Good   7  423
##
##                Accuracy : 0.76
##                  95% CI : (0.4714, 0.7931)
##     No Information Rate : 0.7455
##     P-Value [Acc > NIR] : 0.3125
##
##                   Kappa : 0.2442
##
##  Mcnemar's Test P-Value : 2.8664e-07
##
##             Sensitivity : 0.2500
##             Specificity : 0.9040
##          Pos Pred Value : 0.6818
##          Neg Pred Value : 0.7615
##              Prevalence : 0.2455
##          Detection Rate : 0.0614
##    Detection Prevalence : 0.1106
##       Balanced Accuracy : 0.5770
##
##        'Positive' Class : Bad
```
```